

OS lab4

一、练习1：分配并初始化一个进程控制块（需要编码）

进程控制块是操作系统中实现进程管理的核心数据结构，它系统性地封装了进程的所有关键属性，包括进程状态、标识信息、资源分配、执行上下文和调度参数等关键元数据。

在 `ucore` 操作系统中，`struct proc_struct` 作为进程控制块的具体实现，通过精心设计的字段组织完整记录了进程从创建到终止整个生命周期所需的全部管理信息，为操作系统的进程调度、内存管理、资源分配和状态监控等核心功能提供了必要的数据库，是实现多进程并发执行和环境隔离的技术基石。

（一）进程控制块关键字段分析

在实现初始化过程前，我们需要理解各关键字段的作用：

- 进程状态 (state)：标识进程在当前生命周期中所处的状态
- 进程标识符 (pid)：系统内唯一标识一个进程的数值
- 内核栈指针 (kstack)：指向进程内核栈的指针，用于内核态执行
- 内存管理信息 (mm)：管理进程的用户空间地址映射
- 上下文 (context)：保存进程切换时的寄存器状态
- 陷阱帧 (tf)：保存中断或异常发生时的处理器完整状态
- 页目录表 (pgdir)：指向进程页表的基地址

（二）实现过程

在 `kern/process/proc.c` 文件的 `alloc_proc` 函数中，我们完成了进程控制块的完整初始化工作。

这段代码首先通过 `kmalloc` 动态分配了进程控制块的内存空间，随后系统性地**对其所有关键字段进行了安全初始化**：将进程状态设为未初始化 (`PROC_UNINIT`)，进程ID标记为无效值 (`-1`)，内核栈指针清零，并通过 `memset` 确保上下文环境处于确定状态。同时，父进程指针、内存管理结构和陷阱帧指针均初始化为 `NULL`，页目录设置为引导阶段页表，进程标志和名称数组清零，最后初始化了进程链表和哈希表节点。

这套完整的初始化流程为进程的后续创建和调度奠定了安全可靠的基础，具体的代码如下所示：

```
// alloc_proc - alloc a proc_struct and init all fields of proc_struct
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        // LAB4:EXERCISE1 YOUR CODE
        proc->state = PROC_UNINIT;    // 进程状态初始化为未初始化
        proc->pid = -1;                // 进程ID初始化为-1
        proc->runs = 0;                // 运行次数初始化为0
        proc->kstack = 0;              // 内核栈指针初始化为0
        proc->need_resched = 0;        // 是否需要重新调度初始化为false
        proc->parent = NULL;           // 父进程初始化为NULL
        proc->mm = NULL;               // 内存管理结构初始化为NULL
        memset(&proc->context, 0, sizeof(struct context)); // 上下文初始化为全0
    }
}
```

```

proc->tf = NULL; // 中断帧初始化为NULL
proc->pgdir = boot_pgdir_pa; // 页目录表基地址初始化为boot_pgdir_pa
proc->flags = 0; // 进程标志初始化为0
memset(proc->name, 0, sizeof(proc->name)); // 进程名初始化为空字符串
list_init(&proc->list_link); // 初始化进程链表节点
list_init(&proc->hash_link); // 初始化进程哈希表节点
}
return proc;
}

```

关键字段初始化详细说明：

1.state字段：进程状态初始化为 `PROC_UNINIT`，这是进程生命周期的起始点。该状态表明进程控制块已成功分配内存空间，但进程的完整执行环境尚未建立。在此状态下，进程不具备运行资格，需要经过后续的完整初始化流程才能转换为可运行状态。这种设计确保了进程在完全准备好之前不会被意外调度执行，维护了系统的稳定性。

2.pid字段：进程ID初始化为-1，这是一个特殊的无效标识符。在进程控制块分配阶段，系统尚未为进程分配唯一的身份标识。这个负数值明确表示该PCB尚未完成身份认证，需要在后续的 `do_fork` 过程中通过 `get_pid()` 函数获取一个唯一的正整数PID。这种两阶段分配机制避免了PID冲突，确保了进程标识的系统级唯一性。

3.context字段：使用 `memset` 对整个上下文结构进行清零初始化，这是进程切换安全性的关键保障。上下文结构保存了进程切换时需要的所有寄存器状态。清零操作确保了新进程第一次被调度时具有确定的初始状态，避免了随机内存数据导致的不可预测行为，为进程的首次上下文切换提供了干净的运行环境基础。

4.tf字段：陷阱帧指针初始化为NULL，这是因为新创建的进程尚未经历任何中断或异常处理。陷阱帧用于保存进程被中断时的完整处理器状态，包括所有通用寄存器、程序计数器、状态寄存器等。对于全新的进程，不存在需要恢复的中断现场，因此设置为NULL。当进程第一次被调度执行时，系统会通过其他机制来建立其初始执行环境。

5.pgdir字段：页目录基地址初始化为 `boot_pgdir_pa`，即系统引导阶段使用的页目录物理地址。这一设计体现了内核线程的重要特性——共享内核地址空间。所有内核线程都使用相同的内核页表，它们在内核态执行时共享相同的内存映射关系，包括内核代码段、数据段和设备映射区域。这种共享机制简化了内核线程的内存管理，提高了系统性能，同时保持了地址空间的隔离安全性。

（三）实验结果

编写好我们对应的结构体后，我们在命令行窗口输出 `make qemu` 则会得到如下所示的结果，当我们看到 `alloc_proc()` correct! 的时候，则说明我们的结构体编写正确，可以继续进行下一步的实验。

```

check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!

```

(四) 回答问题

请说明proc_struct中struct context context和struct trapframe *tf成员变量含义和在本实验中的作用是什么？

struct context context:

- 含义：保存进程的上下文信息，主要用于进程切换时保存和恢复处理器的寄存器状态。
- 作用：在进程调度器进行主动的进程切换时，`switch_to` 函数会将当前进程的寄存器状态保存到其context中，然后从目标进程的context中恢复寄存器状态，实现进程的透明切换。context主要包含进程调度相关的必要寄存器，如ra（返回地址）、sp（栈指针）、s0-s11（保存寄存器）等。

struct trapframe *tf:

- 含义：指向陷阱帧的指针，用于保存中断或异常发生时处理器的完整状态。
- 作用：在系统运行过程中，陷阱帧承担着关键的**现场保护与恢复职能**：当中断或异常发生时，它完整保存被中断进程的处理器状态，包括所有寄存器值和程序计数器；在进程首次被调度时，系统通过陷阱帧恢复其初始执行环境；在系统调用过程中，它作为参数传递和返回值保存的媒介；对于新创建的进程，陷阱帧则封装了其初始的执行上下文，包括入口地址和初始寄存器状态，为进程的第一次执行提供完整的启动环境。

两者关键区别：

1、功能定位：

- context 专用于**进程间的主动调度切换**，主要服务于操作系统的进程调度器。当进程因时间片耗尽、主动放弃CPU或优先级调整等原因需要切换时，通过保存和恢复context实现平滑的进程轮转。
- tf 面向**中断/异常等被动现场保护**，处理来自硬件或软件的异步事件。包括系统调用、缺页异常、外设中断等场景，确保这些事件处理完毕后能精确恢复到中断前的执行状态。

2、实现机制：

- context 采用精简优化设计，基于性能考虑**仅保存进程调度必需的核心寄存器集合**，包括返回地址(ra)、栈指针(sp)、帧指针(s0)以及保存寄存器(s1-s11)。这种设计减少了上下文切换的开销，提高了调度效率。
- tf 采用**完整状态保存策略**，基于可靠性要求记录中断发生时处理器的全部状态信息。不仅包含所有通用寄存器(x0-x31)，还包括程序计数器(epc)、状态寄存器(sstatus)、异常原因寄存器(scause)等系统寄存器，确保异常返回时能够完全复原现场。

二、练习2：为新创建的内核线程分配资源（需要编码）

这个实验的目标是深入理解 **ucore** 操作系统在创建新内核线程时的底层实现，也就是弄清楚系统如何为一个新线程分配资源、复制上下文、建立运行环境，并最终将它纳入到内核的进程调度体系中。

(一) 实验思路与流程

在 ucore 里，创建一个新的内核线程最终通过 `kernel_thread()` 完成，而 `kernel_thread()` 会调用 `do_fork()`。`do_fork()` 就是整个实验的核心函数，它主要完成如下几个关键步骤（也是我自己实现的逻辑顺序）：

1.调用 `alloc_proc()`

分配并初始化一个新的 `proc_struct`，也就是进程控制块（PCB）。它相当于操作系统为新线程建立的“档案袋”，里面会记录线程的各种状态、栈、上下文、父子关系等。

```
// 1. 调用 alloc_proc 分配一个新的进程控制块（proc_struct）
if ((proc = alloc_proc()) == NULL)
{
    goto fork_out;
}
```

2.分配内核栈（`setup_kstack()`）

每个进程或线程都需要一个独立的内核栈，用来保存中断时的临时数据、内核调用现场等。这个栈的分配是通过页分配器实现的，保证栈空间连续且在内核态下可访问。

```
// 2. 调用 setup_kstack 为子进程分配内核栈
if (setup_kstack(proc) != 0)
{
    goto bad_fork_cleanup_proc;
}
```

3.复制内存管理信息（`copy_mm()`）

由于当前实验是内核线程（不是用户进程），所以其实不需要复制用户空间的内存管理信息。这一步可以简单返回 0，但我理解这部分逻辑在后续实验中会变得很关键。

```
// 3. 调用 copy_mm 根据 clone_flag 决定是复制还是共享内存空间
if (copy_mm(clone_flags, proc) != 0)
{
    goto bad_fork_cleanup_kstack;
}
```

4.复制上下文和中断帧（`copy_thread()`）

这一步是最有意思的部分，内核通过复制当前进程的 `trapframe`，为新线程建立运行现场。新线程醒来的第一条指令其实是从 `forkret()` 开始执行的，这就相当于线程被“放到CPU上”后从这里跳进去运行。

```
// 4. 调用 copy_thread 设置子进程的中断帧（tf）和执行上下文（context）
copy_thread(proc, stack, tf);
```

5.分配唯一 PID、插入进程链表和哈希表

系统为每个进程分配唯一 PID，`get_pid()` 函数负责保证不会重复。这一步实际上相当于把新建的进程“挂上系统管理的名单”，使它能被查找、调度、唤醒。

```
// 5. 将新建的 proc_struct 插入到哈希表和进程链表中
proc->pid = get_pid();
proc->parent = current;
hash_proc(proc);
list_add(&proc_list, &proc->list_link);
nr_process++;
```

6.唤醒新进程（`wakeup_proc()`）

让新线程从 `PROC_UNINIT` 状态变为 `PROC_RUNNABLE`，表示它已经准备好可以被调度器选中执行。

```
// 6. 调用 wakeup_proc 让新建的子进程变为 RUNNABLE 状态
wakeup_proc(proc);
```

7. 返回新线程的 PID

父进程可以拿到这个 PID，从而在逻辑上与新线程建立联系。

```
// 7. 返回值设为子进程的 pid
ret = proc->pid;
```

我在理解 `do_fork()` 的时候，最开始觉得它只是复制结构体，但后来发现其实它做的是“资源的克隆与链接”工作——不仅仅是复制内存，而是将新线程完整纳入整个调度体系。这种“复制+注册”的模式，体现了 ucore 内核结构的清晰性。

(二) 完整代码展示

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    // LAB4:EXERCISE2 YOUR CODE

    // 1. 调用 alloc_proc 分配一个新的进程控制块 (proc_struct)
    if ((proc = alloc_proc()) == NULL)
    {
        goto fork_out;
    }

    // 2. 调用 setup_kstack 为子进程分配内核栈
    if (setup_kstack(proc) != 0)
    {
        goto bad_fork_cleanup_proc;
    }

    // 3. 调用 copy_mm 根据 clone_flag 决定是复制还是共享内存空间
    if (copy_mm(clone_flags, proc) != 0)
    {
        goto bad_fork_cleanup_kstack;
    }

    // 4. 调用 copy_thread 设置子进程的中断帧 (tf) 和执行上下文 (context)
    copy_thread(proc, stack, tf);

    // 5. 将新建的 proc_struct 插入到哈希表和进程链表中
    proc->pid = get_pid();
    proc->parent = current;
    hash_proc(proc);
}
```

```

list_add(&proc_list, &proc->list_link);
nr_process++;

// 6. 调用 wakeup_proc 让新建的子进程变为 RUNNABLE 状态
wakeup_proc(proc);

// 7. 返回值设为子进程的 pid
ret = proc->pid;

fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

(三) 回答问题

问题：请说明ucore是否做到给每个新fork的线程一个唯一的id？请说明你的分析和理由。

我的分析是：**是的，ucore保证了PID的唯一性。**

理由如下：

1. `get_pid()` 函数中使用了静态变量 `last_pid` 来记录上一次分配的 PID，并在分配时递增。
2. 每次分配后会遍历整个 `proc_list`，确保当前 `last_pid` 没有被占用。
3. 如果发现冲突，就继续递增 PID，直到找到一个未被使用的编号。
4. 当 PID 达到上限 `MAX_PID` 时，会回绕到 1，从头重新寻找。

这种设计虽然简单，但很有效。它确保同一时刻不会出现两个进程拥有相同 PID。而且当旧进程退出后，它的 PID 可以被回收利用，不会无限增长。

我觉得这种机制在小型内核（如 ucore）中已经足够稳定可靠。当然，在更复杂的系统（比如 Linux）中会引入更高效的 PID 分配算法和哈希结构，但原理是一致的。

三、练习3：编写proc_run 函数（需要编码）

（一）函数功能概述

`proc_run` 用于将指定的进程切换到CPU上运行，实现进程上下文切换

（二）详细执行步骤分析

1. 检查进程是否相同

```
if (proc != current)
```

避免不必要的上下文切换开销，如果要切换的进程就是当前正在运行的进程，直接返回，节省性能

2.禁用中断

```
uint32_t flags;  
local_intr_save(flags);
```

- 目的：保证进程切换的原子性
- 实现原理：
 - `local_intr_save(flags)` 保存当前中断状态并禁用中断
 - 防止在切换过程中被中断打断，导致数据不一致
 - `flags` 用于保存原始中断状态，以便后续恢复

3.更新当前进程指针

```
struct proc_struct *prev = current;  
current = proc;
```

其中变量 `prev` 是保存原进程的指针，用于上下文切换，`current` 是全局变量，指向当前运行进程

4.切换页表

```
lsatp(proc->pgdir);
```

每个进程有独立的页表（`proc->pgdir`），切换后MMU使用新进程的地址空间映射，实现进程间的内存隔离。

5.上下文切换

```
switch_to(&(prev->context), &(proc->context));
```

这是最核心的部分，`switch_to` 在 `switch.s` 中实现，用于将当前运行的进程的上下文保存起来，并恢复要切换的进程的上下文，

6.恢复中断

```
local_intr_restore(flags);
```

在 `switch_to` 返回后执行，此时已经在新进程的上下文中执行，恢复原来的中断状态，允许中断。

(三) 代码展示

```
void proc_run(struct proc_struct *proc)  
{  
    if (proc != current)  
    {  
        // 1. 禁用中断  
        uint32_t flags;  
        local_intr_save(flags);  
  
        // 2. 切换当前进程为要运行的进程  
        struct proc_struct *prev = current;  
        current = proc;
```

```

// 3. 切换页表，以便使用新进程的地址空间
lsatp(proc->pgdir);

// 4. 实现上下文切换
switch_to(&(prev->context), &(proc->context));

// 5. 允许中断
local_intr_restore(flags);
}
}

```

(四) 回答问题

在本实验的执行过程中，创建且运行了几个内核线程？

根据实验执行过程和代码分析，在本实验的执行过程中，创建且运行了 2 个内核线程

- 1.第一个内核线程：**idleproc (空闲进程)**，在 `proc_init()` 函数中直接创建，是系统的第一个进程，当没有其他进程可运行时执行，主要工作是在 `cpu_idle()` 中循环，不断检查是否需要调度。
- 2.第二个内核线程：**initproc (初始化进程)**，在 `proc_init()` 中通过 `kernel_thread(init_main, "Hello world!!", 0)` 创建，执行 `init_main` 函数，用来打印进程信息和欢迎消息，为后续用户进程的创建做准备。

四、实验结果

完成代码编写后，编译并运行代码，得到的结果如下图所示：

(一) `make qemu`

```

Special kernel symbols:
  entry 0xc020004a (virtual)
  etext 0xc0203ec4 (virtual)
  edata 0xc0209030 (virtual)
  end   0xc020d4ec (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physcial memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaoffset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
kernel panic at kern/process/proc.c:421:
  process exit!!

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.

```

分析输出结果：

1.内核加载和符号解析

```
Special kernel symbols:
  entry 0xc020004a (virtual)
  etext 0xc0203ec4 (virtual)
  edata 0xc0209030 (virtual)
  end    0xc020d4ec (virtual)
kernel executable memory footprint: 54KB
```

指明了内核入口点：0xc020004a，内核代码段结束于0xc0203ec4，内核数据段结束于0xc0209030，这些都是虚拟地址。最后输出了内核总大小：54KB

2.内存管理系统检查

```
memory management: default_pmm_manager
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
```

我们使用的是默认物理内存管理器，页分配、页目录、启动页目录检查全部通过。

3.内核内存分配器

```
use SLOB allocator
kmalloc_init() succeeded!
```

使用SLOB分配器，内核内存分配初始化成功

4.虚拟内存管理检查

```
check_vma_struct() succeeded!
check_vmm() succeeded.
```

虚拟内存区域结构检查通过，虚拟内存管理器检查通过

5.进程管理初始化

```
alloc_proc() correct!
```

表明**练习1**的**alloc_proc**函数正确实现，进程控制块分配和初始化正常工作

6.中断系统初始化

```
++ setup timer interrupts
```

定时器中断设置完成，为进程调度提供了时间片基础

7.内核线程创建和运行

```
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
```

这是在 `proc_init()` 中通过 `kernel_thread()` 创建的线程2，成功执行 `init_main` 函数，打印了预期的输出信息，**证明练习2和练习3成功：进程创建和切换正常工作**

8. 进程退出处理

```
kernel panic at kern/process/proc.c:421:
process exit!!.
```

当 `initproc` 执行完毕后调用 `do_exit`，当前 `do_exit` 实现只是简单触发 panic，这是预期行为，说明**进程正常执行完毕**

9. 内核调试监控器

```
welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
```

系统进入内核调试监控状态

(二) make grade

```
fault_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc kern/mm/vmm.c + cc ke
rn/process/entry.S + cc kern/process/proc.c + cc kern/process/switch.S + cc kern
/schedule/sched.c + cc libs/hash.c + cc libs/printfmt.c + cc libs/string.c + ld
bin/kernel riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucor
e.img gmake[1]: 离开目录“/home/youth/lab/lab4”
-check alloc proc: OK
-check initproc: OK
Total Score: 30/30
youth@youth-VMware-Virtual-Platform:~/lab/lab4$
```

`check alloc proc` 检查进程分配，表明练习1 通过

`check initproc` 检查初始化进程，表明练习2 和 练习3 通过

最终得分30/30，证明所有要求的功能都正确实现了！

五、扩展练习 Challenge:

(一) 说明语句

`local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是如何实现开关中断的？

在本实验中，`local_intr_save(intr_flag)` 和 `local_intr_restore(intr_flag)` 是 uCore 操作系统中用于**保存、关闭与恢复中断状态**的重要宏。它们的核心原理是通过 **RISC-V 架构下的控制与状态寄存器（CSR）** 操作，来实现对中断的精准控制。下面我从实现层次和执行流程两个方面，对其工作原理进行分析与说明。

1. 宏定义层面的封装

在 `sync.h` 中，这两个宏的定义如下：

```
#define local_intr_save(x) \
do { \
    x = __intr_save(); \
} while (0)

#define local_intr_restore(x) __intr_restore(x);
```

可以看出，`local_intr_save` 使用了 `do-while(0)` 的结构来保证语句的原子性，使其在复杂语句中依然能安全展开；宏会调用内部函数 `__intr_save()`，并将返回的中断状态保存到变量 `x` 中；而 `local_intr_restore` 则直接调用 `__intr_restore()` 来恢复中断状态。我认为这种设计的优势在于它既简洁又安全，同时保持了操作的局部性，不会影响系统中其他部分的中断逻辑。

2.内联函数实现层

这两个宏实际上对应的具体操作定义在 `sync.h` 内部的内联函数中：

```
static inline bool __intr_save(void) {
    if (read_csr(sstatus) & SSTATUS_SIE) {
        intr_disable();
        return 1;
    }
    return 0;
}

static inline void __intr_restore(bool flag) {
    if (flag) {
        intr_enable();
    }
}
```

其工作机制是：

- `__intr_save` 首先读取 `sstatus` 寄存器，检查其中的 `SIE` 位（Supervisor Interrupt Enable，表示中断是否开启）；
 - 如果中断处于开启状态（`SIE = 1`），则立即调用 `intr_disable()` 来关闭中断；
 - 同时返回 `1`，表示原来中断是开启的；
 - 如果中断本就关闭，则直接返回 `0`。
- `__intr_restore` 根据传入的 `flag` 判断是否恢复中断；
 - 当 `flag = 1` 时，调用 `intr_enable()` 重新开启中断；
 - 若 `flag = 0`，则不做任何操作。

换句话说，系统可以根据保存的标志精确恢复到操作前的中断状态，这种“保存-恢复”机制非常符合内核在临界区操作时的安全性需求。

3.中断开关的底层实现

在 `intr.c` 文件中，可以看到在刚刚 `sync.h` 中真正操作中断的函数：

```
void intr_enable(void) { set_csr(sstatus, SSTATUS_SIE); }
void intr_disable(void) { clear_csr(sstatus, SSTATUS_SIE); }
```

- `intr_enable()` 通过设置 `sstatus` 寄存器中的 `SIE` 位来开启中断;
- `intr_disable()` 则清除该位, 禁止中断。

可以理解为这两个函数直接在硬件层面对中断开关进行控制, 而不依赖任何软件标志。

4.RISC-V 架构下的 CSR 操作机制

上面那些寄存器操作最终都依赖于在 `riscv.h` 中定义的宏, 通过内联汇编来完成:

```
#define read_csr(reg) ({ unsigned long __tmp; \
    asm volatile ("csrr %0, " #reg : "=r"(__tmp)); \
    __tmp; })

#define set_csr(reg, bit) ({ unsigned long __tmp; \
    asm volatile ("csrrs %0, " #reg ", %1" : "=r"(__tmp) : "rk"(bit)); \
    __tmp; })

#define clear_csr(reg, bit) ({ unsigned long __tmp; \
    asm volatile ("csrrc %0, " #reg ", %1" : "=r"(__tmp) : "rk"(bit)); \
    __tmp; })
```

- `csrr`: 读取指定 CSR 寄存器;
- `csrrs`: 设置寄存器的某一位;
- `csrrc`: 清除寄存器的某一位。

这些操作都是原子的, 因此可以确保中断状态在切换过程中不会出现竞争条件。

5.整体工作流程总结

综合来看, `local_intr_save` 与 `local_intr_restore` 的完整工作过程如下:

(1) 保存并关闭中断:

- 调用 `__intr_save()`;
- 检查 `sstatus` 寄存器中的 `SIE` 位;
- 若中断已启用, 则调用 `intr_disable()` 清除该位;
- 返回原始状态标志 (1 表示原本启用, 0 表示原本关闭), 并保存在 `intr_flag` 变量中。

(2) 恢复中断:

- 调用 `__intr_restore(intr_flag)`;
- 若 `intr_flag == 1`, 说明原本中断是开启的, 此时调用 `intr_enable()` 重新设置 `SIE` 位;
- 若 `intr_flag == 0`, 则不做任何操作。

6.我的理解与体会

通过这一部分的阅读与分析, 我认为 `local_intr_save` / `local_intr_restore` 的设计思路非常“内核级”。它的目标并不是单纯地“开关中断”, 而是为了**在执行关键内核操作时防止中断打断**, 保证操作的**原子性和一致性**。

此外, 使用 CSR 指令进行直接寄存器操作也体现了 RISC-V 架构在系统级编程上的简洁性和可控性。在后续学习内核同步机制时, 这种保存-恢复的思想也为理解锁机制的实现提供了很好的基础。

(二) 深入理解不同分页模式的工作原理（思考题）

1.get_pte()函数中有两段形式类似的代码，结合sv32，sv39，sv48的异同，解释这两段代码为什么如此相像。

在分析 get_pte() 函数时，我们可以观察到其中包含两段结构高度相似的代码，这两段代码分别负责处理Sv39分页方案中的前两级页表遍历。这种相似性并非偶然，而是深刻反映了RISC-V体系结构下分页系统的设计。

具体的两段代码如下所示：

第一段代码处理第一级页目录（对应VPN[2]）：

```
pde_t *pdep1 = &pgdir[PDX1(1a)];
if (!(*pdep1 & PTE_V)) {
    struct Page *page;
    if (!create || (page = alloc_page()) == NULL) {
        return NULL;
    }
    set_page_ref(page, 1);
    uintptr_t pa = page2pa(page);
    memset(KADDR(pa), 0, PGSIZE);
    *pdep1 = pte_create(page2ppn(page), PTE_U | PTE_V);
}
```

第二段代码处理第二级页目录（对应VPN[1]）：

```
pde_t *pdep0 = &((pte_t *)KADDR(PDE_ADDR(*pdep1)))[PDX0(1a)];
if (!(*pdep0 & PTE_V)) {
    struct Page *page;
    if (!create || (page = alloc_page()) == NULL) {
        return NULL;
    }
    set_page_ref(page, 1);
    uintptr_t pa = page2pa(page);
    memset(KADDR(pa), 0, PGSIZE);
    *pdep0 = pte_create(page2ppn(page), PTE_U | PTE_V);
}
```

代码结构对比分析显示，第一段代码处理第一级页目录（对应VPN[2]），第二段代码处理第二级页目录（对应VPN[1]）。尽管它们处理的页表层级不同，但**核心逻辑完全一致**：首先通过虚拟地址的相应索引段定位页表项，然后检查其有效性，在需要时分配新的物理页，最后完成页表的初始化和设置工作。

这种设计模式的深层原因在于**RISC-V分页方案的统一架构设计**。无论是Sv32（2级页表）、Sv39（3级页表）还是Sv48（4级页表），它们都建立在相同的**多级页表树状结构**之上。虽然各级方案在虚拟地址宽度和分级数量上存在差异：

- Sv32采用32位地址空间和2级页表结构
- Sv39采用39位地址空间和3级页表结构
- Sv48采用48位地址空间和4级页表结构

但每一级页表的处理逻辑保持着本质的一致性。这种一致性体现在四个核心步骤中：

- (1) 通过虚拟地址索引定位当前层级的页表项

- (2) 验证页表项的有效性标志位
- (3) 在必要时为缺失的页表分配物理内存
- (4) 初始化新分配的页表并设置正确的权限标志

除此以外，从硬件抽象与代码复用的角度来看，这种设计体现了优秀的工程思想。代码中**唯一变化的参数是输入的基地址和使用的索引，而核心的页表分配和初始化逻辑得以完全复用**。这种"自相似"的架构使得扩展支持更复杂的分页方案（如Sv48）变得异常简单——只需添加遵循相同模式的额外代码段即可。

总而言之，这两段代码的相似性绝非简单的代码重复，而是对RISC-V多级页表规整架构的精准映射。它体现了"单一页表层级处理"这一通用算法在实践中的实现，既保证了代码的可维护性，又为系统的可扩展性奠定了坚实基础。

2.目前get_pte()函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

我认为当前 `get_pte()` 函数将页表项查找与分配功能合并的写法是好的，但在特定场景下有必要将两个功能拆开。

(1) 当前合并设计的优势分析

从操作系统内核的实际应用场景出发，`get_pte()` 函数采用查找与分配功能合并的设计具有多重工程实践优势。

首先，接口语义与使用场景高度匹配。接口语义的精准匹配体现在其"确保获取"的原子操作特性。这种设计完美契合了内核中建立内存映射的核心需求模式。以 `page_insert` 函数的典型应用为例：

```
// 在page_insert函数中的典型用法
pte_t *ptep = get_pte(pgdir, la, 1);
if (ptep == NULL) {
    return -E_NO_MEM;
}
```

调用方的主要诉求是确保获得有效的页表项以完成地址映射，而非关心中间页表层的构建细节。这种"一站式"解决方案提供了恰当的抽象层次，显著简化了调用方的逻辑复杂度。

其次，性能优化效果明显。合并实现在单次页表遍历过程中同步完成查找与分配操作。如果采用分离设计，查找函数定位缺失点后，分配函数需要重新遍历相同路径，造成不必要的性能开销。当前实现通过统一流程处理所有中间层级，最大限度地提升了执行效率。

第三，状态一致性得到可靠保障。内聚的分配逻辑：

```
// 内嵌的分配逻辑确保了正确性
if (!create || (page = alloc_page()) == NULL) {
    return NULL;
}
set_page_ref(page, 1);
memset(KADDR(pa), 0, PGSIZE);
*ptep = pte_create(...);
```

内聚的分配逻辑集中管理了引用计数、页面初始化和权限设置等关键操作，形成了完整的事务边界。这种封装有效防止了调用方因处理分布式状态而可能引入的竞态条件和一致性错误，提升了系统整体的可靠性。

(2) 拆分开的理论必要性

尽管合并设计优势明显，但在以下场景中功能拆分具有重要价值：

单一职责原则的严格遵循要求每个函数仅承担明确定义的单一功能。理想的设计方案应当建立清晰的职责边界：

```
// 纯查询接口，无副作用
pte_t *pte_lookup(pde_t *pgdir, uintptr_t la);

// 分配接口，负责创建缺失的页表
pte_t *pte_ensure(pde_t *pgdir, uintptr_t la);
```

这种分离使得代码的意图更加明确，降低了认知复杂度，并提高了单元测试的可行性。

接口粒度的精确控制是拆分的另一重要收益。合并设计中的 `create` 参数导致接口语义存在二义性：调用方可能无意中触发分配操作，或者相反，在需要分配时未能正确设置参数。分离的接口通过函数名明确表达意图，消除了此类隐式行为的风险。

错误处理的精细化在拆分后得以实现。查找失败（映射不存在）和分配失败（内存不足）是性质完全不同的异常情况。合并设计统一返回 `NULL`，丢失了具体的错误信息。分离方案允许返回差异化的错误码，使调用方能够根据具体失败原因采取相应的恢复策略。

结论：在当前ucore教学系统中，合并设计因其简洁性和高效性是合理选择。但在大型生产级系统中，基于单一职责原则的功能拆分更能满足长期维护和扩展的需求。建议采用渐进式重构策略，在保持现有接口的同时逐步推进架构优化。